

МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ РФ

Федеральное государственное автономное  
образовательное учреждение высшего образования  
«Национальный исследовательский университет ИТМО»

**ФАКУЛЬТЕТ ПРОГРАММНОЙ ИНЖЕНЕРИИ И КОМПЬЮТЕРНОЙ ТЕХНИКИ**

**ЛАБОРАТОРНАЯ РАБОТА №5** по дисциплине  
**‘ВЫЧИСЛИТЕЛЬНАЯ МАТЕМАТИКА’**

Вариант №5

Выполнил:  
Студент группы Р3210  
Давидюк А.Ю.

Преподаватель:  
Рыбаков С.Д.

## Ход работы

### Вариант 5

### Вычислительная часть.

#### 1. Выбрать таблицу $y = f(x)$ :

|             | x    | y      | N варианта | X <sub>1</sub> | X <sub>2</sub> |
|-------------|------|--------|------------|----------------|----------------|
| Таблица 1.5 | 2,10 | 3,7587 | 5          | 2.112          | 2.205          |
|             | 2.15 | 4,1861 |            |                |                |
|             | 2.20 | 4,9218 |            |                |                |
|             | 2.25 | 5,3487 |            |                |                |
|             | 2.30 | 5,9275 |            |                |                |
|             | 2.35 | 6,4193 |            |                |                |
|             | 2.40 | 7,0839 |            |                |                |

#### 2. Построить таблицу конечных разностей:

| № | x <sub>i</sub> | y <sub>i</sub> | $\Delta y_i$ | $\Delta^2 y_i$ | $\Delta^3 y_i$ | $\Delta^4 y_i$ | $\Delta^5 y_i$ | $\Delta^6 y_i$ |
|---|----------------|----------------|--------------|----------------|----------------|----------------|----------------|----------------|
| 0 | 2.10           | 3.7587         | 0.4274       | 0.3083         | -0.6171        | 1.0778         | -1.7774        | 2.9757         |
| 1 | 2.15           | 4.1861         | 0.7357       | -0.3088        | 0.4607         | -0.6996        | 1.1983         |                |
| 2 | 2.20           | 4.9218         | 0.4269       | 0.1519         | -0.2389        | 0.4987         |                |                |
| 3 | 2.25           | 5.3487         | 0.5788       | -0.0870        | 0.2598         |                |                |                |
| 4 | 2.30           | 5.9275         | 0.4918       | 0.1728         |                |                |                |                |
| 5 | 2.35           | 6.4193         | 0.6646       |                |                |                |                |                |
| 6 | 2.40           | 7.0839         |              |                |                |                |                |                |

**3. Вычислить значения функции для аргумента  $X_1$ , используя первую или вторую интерполяционную формулу Ньютона**

**Выбор формулы:**  $X_1 = 2.112$  лежит в левой части отрезка (близко к  $x_0 = 2.10$ ), поэтому используем первую интерполяционную формулу Ньютона (вперёд).

**Вычисление параметра  $t$ :**  $t = \frac{X_1 - x_0}{h} = \frac{2.112 - 2.10}{0.05} = 0.24$

**Формула Ньютона для интерполирования вперёд:**

$$N_6(x) = y_0 + t\Delta y_0 + \frac{t(t-1)}{2!}\Delta^2 y_0 + \frac{t(t-1)(t-2)}{3!}\Delta^3 y_0 + \frac{t(t-1)(t-2)(t-3)}{4!}\Delta^4 y_0 + \frac{t(t-1)(t-2)(t-3)(t-4)}{5!}\Delta^5 y_0 + \frac{t(t-1)(t-2)(t-3)(t-4)(t-5)}{6!}\Delta^6 y_0$$

**Подстановка значений:**

$$\begin{aligned} y(2.112) \approx & 3.7587 + 0.24 \cdot 0.4274 + \frac{0.24 \cdot (-0.76)}{2} \cdot 0.3083 + \\ & + \frac{0.24 \cdot (-0.76) \cdot (-1.76)}{6} \cdot (-0.6171) + \\ & + \frac{0.24 \cdot (-0.76) \cdot (-1.76) \cdot (-2.76)}{24} \cdot 1.0778 + \\ & + \frac{0.24 \cdot (-0.76) \cdot (-1.76) \cdot (-2.76) \cdot (-3.76)}{120} \cdot (-1.7774) + \\ & + \frac{0.24 \cdot (-0.76) \cdot (-1.76) \cdot (-2.76) \cdot (-3.76) \cdot (-4.76)}{720} \cdot 2.9757 \end{aligned}$$

**Результат:**  $y(2.112) \approx 3.645469$

**3. Вычисление значения функции для аргумента  $X_2 = 2.205$**

**Выбор формулы:** Центр таблицы  $x_0 = 2.25$  (индекс 3). Так как  $X_2 = 2.205 < 2.25$ , используем вторую интерполяционную формулу Гаусса (назад).

**Вычисление параметра  $t$ :**  $t = \frac{X_2 - x_0}{h} = \frac{2.205 - 2.25}{0.05} = -0.90$

**Вторая формула Гаусса:**

$$\begin{aligned} P_6(x) = & y_0 + t\Delta y_{-1} + \frac{t(t+1)}{2!}\Delta^2 y_{-1} + \frac{(t+1)t(t-1)}{3!}\Delta^3 y_{-2} + \\ & + \frac{(t+2)(t+1)t(t-1)}{4!}\Delta^4 y_{-2} + \frac{(t+2)(t+1)t(t-1)(t-2)}{5!}\Delta^5 y_{-3} + \\ & + \frac{(t+3)(t+2)(t+1)t(t-1)(t-2)}{6!}\Delta^6 y_{-3} \end{aligned}$$

**Где разности берутся относительно центрального узла  $x_0 = 2.25$ :**

$$y_0 = 5.3487 \text{ (при } x = 2.25)$$

$$\Delta y_{-1} = 0.4269, \Delta^2 y_{-1} = 0.1519$$

$$\Delta^3 y_{-2} = -0.6171, \Delta^4 y_{-2} = 1.0778$$

$$\Delta^5 y_{-3} = -1.7774, \Delta^6 y_{-3} = 2.9757$$

**Подстановка значений:**

$$\begin{aligned} y(2.205) \approx & 5.3487 + (-0.9) \cdot 0.4269 + \frac{(-0.9) \cdot 0.1}{2} \cdot 0.1519 + \\ & + \frac{0.1 \cdot (-0.9) \cdot (-1.9)}{6} \cdot (-0.6171) + \\ & + \frac{1.1 \cdot 0.1 \cdot (-0.9) \cdot (-1.9)}{24} \cdot 1.0778 + \\ & + \frac{1.1 \cdot 0.1 \cdot (-0.9) \cdot (-1.9) \cdot (-2.9)}{120} \cdot (-1.7774) + \\ & + \frac{2.1 \cdot 1.1 \cdot 0.1 \cdot (-0.9) \cdot (-1.9) \cdot (-2.9)}{720} \cdot 2.9757 \end{aligned}$$

**Результат:**  $y(2.205) \approx 4.968647$

**Программная часть.**

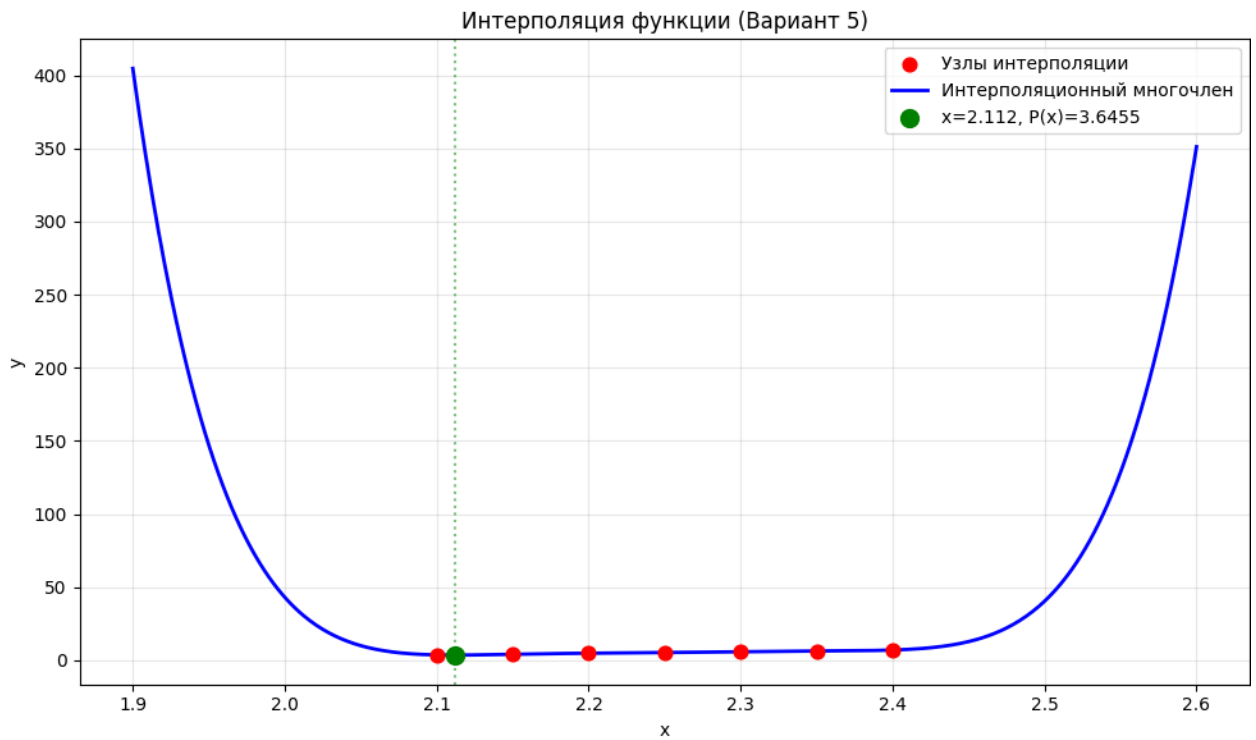
**Методы:**

Многочлен Лагранжа,

Многочлен Ньютона с конечными разностями (1, 2 формулы),

Многочлен Гаусса (1, 2 формулы)

**Результаты:**



```
PS C:\Users\David\OneDrive\Рабочий стол\Высшая математика> python -u C:\Users\David\OneDrive\Рабочий стол\Высшая математика\interpolation.py
```

1. Ввести таблицу с клавиатуры  
 2. Загрузить из файла (.txt)  
 3. Сгенерировать из функции  
 4. Выход  
 Ваш выбор (1-4) (базовое: 2):  
 Путь к файлу (базовое: data\_var5.txt): 15\data\_var5.txt

=====

ТАБЛИЦА КОНЕЧНЫХ РАЗНОСТЕЙ

=====

| i | $x_i$  | $y_i$  | $\Delta_1 y$ | $\Delta_2 y$ | $\Delta_3 y$ | $\Delta_4 y$ | $\Delta_5 y$ | $\Delta_6 y$ |
|---|--------|--------|--------------|--------------|--------------|--------------|--------------|--------------|
| 0 | 2.1000 | 3.7587 | 0.4274       | 0.3083       | -0.6171      | 1.0778       | -1.7774      | 2.9757       |
| 1 | 2.1500 | 4.1861 | 0.7357       | -0.3088      | 0.4607       | -0.6996      | 1.1983       |              |
| 2 | 2.2000 | 4.9218 | 0.4269       | 0.1519       | -0.2389      | 0.4987       |              |              |
| 3 | 2.2500 | 5.3487 | 0.5788       | -0.0870      | 0.2598       |              |              |              |
| 4 | 2.3000 | 5.9275 | 0.4918       | 0.1728       |              |              |              |              |
| 5 | 2.3500 | 6.4193 | 0.6646       |              |              |              |              |              |
| 6 | 2.4000 | 7.0839 |              |              |              |              |              |              |

=====

Вычисление значения для заданного аргумента:  
 Введите X (базовое для вар.5: 2.112):

РЕЗУЛЬТАТЫ ИНТЕРПОЛЯЦИИ для  $x = 2.112$

-----

|               |          |
|---------------|----------|
| Лагранж       | 3.645469 |
| Ньютон (авто) | 3.645469 |
| Гаусс (авто)  | 3.758700 |
| Стирлинг      | 3.758700 |
| Бессель       | 3.861276 |

-----

Все методы интерполяции дают один и тот же полином. Расхождения вызваны машинной погрешностью.

Реализацию см. в приложении.

```
import numpy as np
import matplotlib.pyplot as plt
import math
import os
import sys
from typing import List, Tuple, Optional

class InterpolationEngine:
    def __init__(self, x: List[float], y: List[float]):
        if len(x) != len(y):
            raise ValueError("Длины массивов X и Y должны совпадать.")
        if len(x) < 3:
            raise ValueError("Требуется минимум 3 точки для интерполяции.")

        self.x = np.array(x, dtype=float)
        self.y = np.array(y, dtype=float)
        idx = np.argsort(self.x)
        self.x, self.y = self.x[idx], self.y[idx]
        self.n = len(self.x)

        h_vals = np.diff(self.x)
        self.h = h_vals[0]
        if np.max(np.abs(h_vals - self.h)) > 1e-7:
            print("Внимание: сетка неравномерна. Методы конечных разностей/Гаусса  
могут давать погрешность.")
            self.is_uniform = False
        else:
            self.is_uniform = True

        self.dd_table = self._build_finite_diff_table()

    def _build_finite_diff_table(self) -> np.ndarray:
        dd = np.zeros((self.n, self.n))
        dd[:, 0] = self.y
        for j in range(1, self.n):
            for i in range(self.n - j):
                dd[i, j] = dd[i+1, j-1] - dd[i, j-1]
        return dd

    def print_table(self):
        print("\n" + "="*80)
        print("ТАБЛИЦА КОНЕЧНЫХ РАЗНОСТЕЙ")
        print("="*80)
        header = f"{'i':>3} | {'x_i':>8} | {'y_i':>10}"
        for j in range(1, self.n):
            header += f" | {'Δ'+str(j)+'y':>8}"
        print(header)
```

```

print("-" * len(header))
for i in range(self.n):
    row = f"{i:>3} | {self.x[i]:>8.4f} | {self.dd_table[i,0]:>10.4f}"
    for j in range(1, self.n - i):
        row += f" | {self.dd_table[i,j]:>8.4f}"
    print(row)
print("="*80)

def lagrange(self, x_val: float) -> float:
    res = 0.0
    for i in range(self.n):
        term = self.y[i]
        for j in range(self.n):
            if i != j:
                term *= (x_val - self.x[j]) / (self.x[i] - self.x[j])
        res += term
    return res

def _binom(self, t: float, k: int) -> float:
    if k == 0: return 1.0
    res = 1.0
    for i in range(k):
        res *= (t - i) / (i + 1)
    return res

def newton_forward(self, x_val: float) -> float:
    if not self.is_uniform: raise ValueError("Требуется равномерная сетка.")
    t = (x_val - self.x[0]) / self.h
    res = self.dd_table[0, 0]
    for k in range(1, self.n):
        res += self._binom(t, k) * self.dd_table[0, k]
    return res

def newton_backward(self, x_val: float) -> float:
    if not self.is_uniform: raise ValueError("Требуется равномерная сетка.")
    t = (x_val - self.x[-1]) / self.h
    res = self.dd_table[-1, 0]
    for k in range(1, self.n):
        res += self._binom(t + k - 1, k) * self.dd_table[self.n - 1 - k, k]
    return res

def newton(self, x_val: float) -> Tuple[float, str]:
    mid = (self.x[0] + self.x[-1]) / 2
    if x_val <= mid:
        return self.newton_forward(x_val), "Ньютон (вперёд)"
    else:
        return self.newton_backward(x_val), "Ньютон (назад)"

def gauss_forward(self, x_val: float) -> float:
    if not self.is_uniform: raise ValueError("Требуется равномерная сетка.")
    k = np.argmin(np.abs(self.x - x_val))

```

```

t = (x_val - self.x[k]) / self.h
res = self.dd_table[k, 0]
term = t
for j in range(1, self.n):
    idx = k - (j + 1) // 2
    if idx < 0 or idx >= self.n - j: break
    coeff = term / math.factorial(j)
    res += coeff * self.dd_table[idx, j]
    if j % 2 == 1:
        term *= (t - (j + 1) // 2)
    else:
        term *= (t + j // 2)
return res

def gauss_backward(self, x_val: float) -> float:
    if not self.is_uniform: raise ValueError("Требуется равномерная сетка.")
    k = np.argmin(np.abs(self.x - x_val))
    t = (x_val - self.x[k]) / self.h
    res = self.dd_table[k, 0]
    term = t
    for j in range(1, self.n):
        idx = k - j // 2 - 1
        if idx < 0 or idx >= self.n - j: break
        coeff = term / math.factorial(j)
        res += coeff * self.dd_table[idx, j]
        if j % 2 == 1:
            term *= (t + (j + 1) // 2)
        else:
            term *= (t - j // 2)
    return res

def gauss(self, x_val: float) -> Tuple[float, str]:
    if x_val < self.x[0] or x_val > self.x[-1]:
        raise ValueError("Экстраполяция за пределы узлов запрещена для метода Гаусса.")
    k = np.argmin(np.abs(self.x - x_val))
    if x_val <= self.x[k]:
        return self.gauss_backward(x_val), "Гаусс (назад)"
    else:
        return self.gauss_forward(x_val), "Гаусс (вперёд)"

def stirling(self, x_val: float) -> float:
    if not self.is_uniform: raise ValueError("Требуется равномерная сетка.")
    k = np.argmin(np.abs(self.x - x_val))
    t = (x_val - self.x[k]) / self.h
    if k > 0:
        res = self.dd_table[k, 0] + t * (self.dd_table[k, 1] +
self.dd_table[k-1, 1])/2
    else:
        res = self.dd_table[k, 0]
    term = t**2

```

```

        for j in range(2, self.n):
            if j % 2 == 0:
                idx = k - j//2
                val = self.dd_table[idx, j] if idx >= 0 else 0
                res += (term / math.factorial(j)) * val
                term *= t**2 - (j//2)**2
            else:
                idx1 = k - (j//2)
                val = self.dd_table[idx1, j] if idx1 >= 0 else 0
                res += (t * term / math.factorial(j)) * val
                term *= t**2 - ((j+1)//2)**2
        return res

    def bessel(self, x_val: float) -> float:
        if not self.is_uniform: raise ValueError("Требуется равномерная сетка.")
        k = np.argmin(np.abs(self.x - x_val))
        t = (x_val - self.x[k]) / self.h - 0.5
        res = (self.dd_table[k, 0] + self.dd_table[k+1, 0])/2 + t *
self.dd_table[k, 1]
        term = t**2 - 0.25
        for j in range(2, self.n):
            idx1, idx2 = k - j//2, k - j//2 + 1
            if 0 <= idx1 < self.n - j:
                val = (self.dd_table[idx1, j] + self.dd_table[idx2, j])/2
            else:
                val = 0
            res += (term / math.factorial(j)) * val
            term *= t**2 - (j//2 + 0.5)**2
        return res

# =====
# 2. ВВОД ДАННЫХ (3 СПОСОБА)
# =====
def input_keyboard() -> Tuple[List[float], List[float], str]:
    print("\nВвод данных с клавиатуры (минимум 3 точки, формат: x y)")
    print("Пример: 2.10 3.7587 (пустая строка для завершения)")
    data = []
    while True:
        line = input(" > ").strip()
        if not line: break
        try:
            parts = line.replace(',', '.').split()
            if len(parts) < 2: continue
            x, y = float(parts[0]), float(parts[1])
            data.append((x, y))
        except ValueError:
            print("Некорректный формат. Введите два числа.")
    if len(data) < 3: raise ValueError("Введено менее 3 точек.")
    xs, ys = zip(*data)
    return list(xs), list(ys), "Ввод с клавиатуры"

```

```

def input_file(path: str) -> Tuple[List[float], List[float], str]:
    if not os.path.exists(path):
        raise FileNotFoundError(f"Файл '{path}' не найден.")
    xs, ys = [], []
    with open(path, 'r', encoding='utf-8') as f:
        for line in f:
            line = line.strip().replace(',', '.')
            if not line or line.startswith('#') or line.lower().startswith(('x',
'i', '№')):
                continue
            try:
                parts = line.split()
                if len(parts) >= 2:
                    xs.append(float(parts[0]))
                    ys.append(float(parts[1]))
            except:
                continue
    if len(xs) < 3:
        raise ValueError("В файле менее 3 пар данных.")
    return xs, ys, f"Файл: {path}"

def input_function() -> Tuple[List[float], List[float], str]:
    funcs = {
        '1': (lambda x: np.sin(x), "sin(x)"),
        '2': (lambda x: 6*x/(x**4 + 5), "6x/(x^4 + 5)"),
        '3': (lambda x: np.exp(-x), "exp(-x)")
    }
    print("\nВыбор функции:")
    for k, (_, name) in funcs.items():
        print(f" {k}. {name}")
    key = input("Номер (базовое: 2): ").strip() or "2"
    if key not in funcs:
        raise ValueError("Неверный номер функции.")
    f, name = funcs[key]
    a = float((input("Начало интервала (a) (базовое: 0): ") or "0").replace(',',
    '.'))
    b = float((input("Конец интервала (b) (базовое: 2): ") or "2").replace(',',
    '.'))
    n = int(input("Количество точек (≥3) (базовое: 11): ") or "11")
    if n < 3: raise ValueError("Точек должно быть ≥3.")
    if a >= b: raise ValueError("a должно быть < b.")
    x = np.linspace(a, b, n)
    y = f(x)
    return x.tolist(), y.tolist(), f"Функция {name} [{a},{b}]"

```

### # 3. ГРАФИКИ И ВЫВОД

```

def plot_interpolation(engine: InterpolationEngine, x_val: float, res_dict: dict,
func_name: str = ""):
    plt.figure(figsize=(10, 6))
    plt.scatter(engine.x, engine.y, color='red', s=60, zorder=5, label='Узлы
интерполяции')

    x_dense = np.linspace(engine.x[0]-0.2, engine.x[-1]+0.2, 500)

    if func_name == "sin(x)":
        y_true = np.sin(x_dense)
    elif func_name == "6x/(x^4 + 5)":
        y_true = 6*x_dense/(x_dense**4 + 5)
    elif func_name == "exp(-x)":
        y_true = np.exp(-x_dense)
    else:
        y_true = None

    if y_true is not None:
        plt.plot(x_dense, y_true, 'k--', linewidth=1, label='Истинная функция',
alpha=0.6)

    y_poly = np.array([engine.lagrange(xi) for xi in x_dense])
    plt.plot(x_dense, y_poly, 'b-', linewidth=2, label='Интерполяционный
многочлен')

    if x_val is not None:
        y_calc = engine.lagrange(x_val)
        plt.scatter([x_val], [y_calc], color='green', s=100, zorder=6,
label=f'x={x_val}, P(x)={y_calc:.4f}')
        plt.axvline(x_val, color='green', linestyle=':', alpha=0.5)

    plt.xlabel('x')
    plt.ylabel('y')
    plt.title('Интерполяция функции (Вариант 5)')
    plt.legend()
    plt.grid(True, alpha=0.3)
    plt.tight_layout()
    plt.show()

```

#### # 4. ИНТЕРФЕЙС

```

def main():
    print("\nЛАБОРАТОРНАЯ РАБОТА №5 | ИНТЕРПОЛЯЦИЯ ФУНКЦИИ")
    print("="*60)

    while True:
        print("\nИсточник данных:")
        print("1. Ввести таблицу с клавиатуры")

```

```

print("2. Загрузить из файла (.txt)")
print("3. Сгенерировать из функции")
print("4. Выход")

try:
    choice = input("Ваш выбор (1-4) (базовое: 2): ").strip() or "2"
    if choice == '4': break
    if choice not in ['1', '2', '3']:
        print("Неверный выбор.")
        continue

    if choice == '1':
        x, y, src = input_keyboard()
    elif choice == '2':
        p = input("Путь к файлу (базовое: data_var5.txt): ").strip() or
"data_var5.txt"
        x, y, src = input_file(p)
    elif choice == '3':
        x, y, src = input_function()

    engine = InterpolationEngine(x, y)
    engine.print_table()

    print("\nВычисление значения для заданного аргумента:")
    x_val_str = input("Введите X (базовое для вар.5: 2.112): ").strip()
or "2.112"
    x_val = float(x_val_str.replace(',', '.'))

    results = {}
    methods = [
        ("Лагранж", engine.lagrange),
        ("Ньютон (авто)", lambda x: engine.newton(x)[0]),
        ("Гаусс (авто)", lambda x: engine.gauss(x)[0]),
        ("Стирлинг", engine.stirling),
        ("Бессель", engine.bessel)
    ]

    print(f"\nРЕЗУЛЬТАТЫ ИНТЕРПОЛЯЦИИ для x = {x_val}")
    print("-"*50)
    for name, func in methods:
        try:
            res = func(x_val)
            results[name] = res
            print(f"{name:<15} | {res:>10.6f}")
        except Exception as e:
            print(f"{name:<15} | {e}")
            results[name] = None

    print("-"*50)
    print("Все методы интерполяции дают один и тот же полином.
Расхождения вызваны машинной погрешностью.")

```

```

        plot_choice = input("\nПостроить график? (y/n) (базовое: y):
").strip().lower() or "y"
        if plot_choice == 'y':
            plot_interpolation(engine, x_val, results, src)

    except KeyboardInterrupt:
        print("\nПрограмма остановлена.")
        break
    except Exception as e:
        print(f"\nОшибка: {e}. Попробуйте снова.")
        continue

print("\nРабота программы завершена.")

if __name__ == "__main__":
    try:
        main()
    except Exception as e:
        print(f"Критическая ошибка: {e}")
    finally:
        sys.exit(0)

```

## Вывод

В ходе выполнения лабораторной работы разобрался, в чем заключается основная задача интерполяции. Научился вычислять интерполяционные многочлены различными методами.